

第 9 章 GitHub 网页功能导览

本章目标：认识 GitHub 网页上除了「存代码」之外的实用功能，知道什么场景用什么。

9.1 仓库标签栏全景

打开任意仓库主页，顶部有一排标签，按重要性排序：

标签	作用	使用频率
Code	文件列表、克隆地址（默认页）	每天
Issues	问题跟踪：报 bug、提需求、讨论	经常
Pull requests	合并申请单列表	经常
Actions	自动化流水线（测试、部署）	看结果时
Projects	看板式任务管理	视需要
Wiki	项目文档	偶尔
Settings	仓库设置（改名、删除、分支保护等）	偶尔

9.2 Issues：报 bug 和提需求的地方

Issues 是仓库的「问题台账」，用来：

- **报 bug**：「登录按钮点了没反应」
- **提需求**：「希望支持深色模式」
- **发起讨论**：「要不要升级到 Vue 3？」

创建一个 Issue：Issues 标签 → **New issue** → 填写标题和正文 → **Submit new issue**。

好 Issue 的四要素（照着写，别人才能帮你）：

要素	示例
现象	点击登录按钮，页面无任何反应
复现步骤	1. 打开登录页 2. 输入账号密码 3. 点击登录
期望结果	跳转到个人主页
环境	Windows 11, Chrome 120, 项目版本 v1.2.0

Issue 与代码联动：

- 在提交信息或 PR 描述里写 `fix #12`，PR 合并后 **#12 号 Issue 自动关闭**

- 在 Issue 里用 @用户名 可以提醒某个人来看

9.3 Actions: 自动化流水线

Actions 是 GitHub 内置的**自动化机器人**：每当触发条件满足（比如有人 push），就自动执行你写好的任务。

典型用途：

场景	说明
自动测试	每次 push 自动跑测试，PR 里显示红叉/绿勾
自动部署	代码推到 main 就自动发布网站
定时任务	每天凌晨自动抓数据、发报告
自动构建	打包成安装包 / Docker 镜像

怎么用：在仓库里建一个文件 `.github/workflows/任意名字.yml`，写入任务定义。例如「每次 push 跑 Python 测试」：

```
name: 自动测试
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - run: pip install -r requirements.txt
      - run: pytest
```

提交后：

- 每次 push 自动触发，结果在 **Actions** 标签查看
- 每个 PR 底部会显示一个 **✓ All checks have passed** 或红色 ×
- 失败时 GitHub 会给你**发邮件通知**

新手现在知道「它是干什么的」就够。进阶玩法：配合分支保护规则「测试不通过就不允许合并」，见 9.7。

9.4 GitHub Pages: 免费建一个网站

GitHub 免费送每个账号**静态网站托管**服务，地址形如 `https://你的用户名.github.io/仓库名/`。

能放什么：纯 HTML/CSS/JS、Vue/React 打包后的静态文件、Markdown 博客等（不能放需要后端的动态网站）。

启用步骤：

- 1. 准备一个仓库，里面放 `index.html`（网站首页）
- 2. 仓库 → **Settings** → 左侧 **Pages**
- 3. **Build and deployment** → **Source** 选 `Deploy from a branch`
- 4. **Branch** 选 `main`，目录选 `/ (root)`，点 **Save**
- 5. 等 1~2 分钟，访问给出的网址即可看到你的网站

每个人还有一个专属空间：创建一个名为 `你的用户名.github.io` 的仓库，它就是你的个人主页，地址就是 `https://你的用户名.github.io/`。

9.5 搜索：GitHub 是一座宝库

顶部搜索框输入关键词，按分类浏览结果。进阶筛选语法（直接敲在搜索框里）：

语法	含义
<code>python in:name</code>	名字里含 python
<code>stars:>1000</code>	星标数大于 1000
<code>language:typescript</code>	指定语言
<code>pushed:>2026-01-01</code>	最近仍在更新
<code>awesome in:name</code>	找「awesome 合集」——各领域的资源清单

`awesome-python`、`awesome-cpp`、`awesome-chatgpt-prompts`任何领域都有对应的 awesome 仓库，是**入行找学习资源最快的入口**。

9.6 Star / Watch / Fork 的区别

按钮	作用	类比
Star	收藏，表示喜欢/支持	点赞 + 书签
Watch	订阅通知，仓库有动态会通知你	关注
Fork	复制一份到自己名下	复印一份带走

9.7 分支保护规则（进阶但很实用）

防止自己（或队友）手滑把坏代码直接推进 main：

1. 仓库 → **Settings** → **Branches** → **Add branch protection rule**（或 Add classic branch protection rule）
2. **Branch name pattern** 填 `main`
3. 勾选：
 - **Require a pull request before merging**：必须走 PR 才能合入 main
 - **Require status checks to pass before merging**：Actions 测试通过才允许合并
4. 保存

从此所有改动都必须经过「PR + 审查 + 测试通过」才能进 main——这是**企业级项目的标准做法**，个人项目也强烈建议开启。

9.8 把 GitHub 当作品集

对求职者来说，GitHub 主页是**免费的简历**：

- **个人主页装修**：创建一个名为 `你的用户名` 的仓库（与用户名同名），里面的 README 会直接显示在你的 GitHub 个人主页上。写上简介、技能、项目链接、联系方式
- **仓库的 README 认真写**：面试官点进仓库第一眼看到的就是它
- **给仓库配 License**：开源项目建议选 MIT（最宽松常用）；纯个人练习可选不公开
- **提交历史规律**：连续的小绿点（Contributions 日历）能体现持续学习的习惯

9.9 不想用命令行？两个官方工具

工具	适合	获取
GitHub Desktop	怕命令行的纯新手，图形界面完成 add/commit/push	https://desktop.github.com
GitHub CLI (gh)	喜欢命令行的进阶用户，在终端直接建仓库、发 PR	winget install --id GitHub.cli 后运行 <code>gh auth login</code>

建议：即使有图形工具，也**先按本书把命令行流程走熟**——命令行是 Git 的「原版界面」，几乎所有教程、报错解答都以它为准。图形工具只是把同样的命令包装成按钮。

9.10 本章小任务

- [] 给自己的一个仓库创建一条 Issue（比如记录一个想做的功能）
- [] 给仓库开一个 GitHub Pages，看到自己的网页上线

- [] 用搜索语法 `stars:>1000 language:python` 找到 3 个感兴趣的仓库并 Star
- [] 创建一个与用户名同名的仓库，写一份个人主页 README

下一章：附录——命令速查表和常见报错对照表，出问题先翻它。